

Rezolvare Completă și Detaliată

Examenul Național pentru Definitivare în Învățământ

Anul 2014 - Informatică

Cuprins

I. Rezolvare Definitivat Varianta 1 (14 iulie 2014)	2
SUBIECTUL I (30 de puncte)	2
1. Grafuri Hamiltoniene	2
2. Structura sistemelor de calcul: Sistemul de operare	2
3. Programare: Secvență de elemente cu cifre egale (distinct = 1)	3
a) Subprogramul distinct	3
b) Programul pentru secvența maximă	5
SUBIECTUL al II-lea (30 de puncte)	5
1. Aspecte ale proiectării didactice pentru conținutul A: Structuri repetitive	5
a) Tipuri de lecție și momente ale lecției	5
b) Activități de învățare	5
2. Test oral pentru conținutul A: Structuri repetitive	6
SUBIECTUL al III-lea (30 de puncte)	6
Învățarea: concept, condiții interne și condiții externe	6
II. Rezolvare Definitivat Informatică de gestiune Varianta 1 (14 iulie 2014)	8
SUBIECTUL I (30 de puncte)	8
1. Imprimanta în arhitectura sistemului de calcul	8
2. Sistemul de operare	8
3. Subprogramul distinct și programul cerut	8
4. Bază de date pentru companie de telefonie	9
SUBIECTUL al II-lea (30 de puncte)	10
1. Proiectare didactică pentru structuri repetitive	10
2. Test oral pentru structuri repetitive	10
SUBIECTUL al III-lea (30 de puncte)	10

I. Rezolvare Definitivat Varianta 1 (14 iulie 2014)

[Subiect PDF](file:

SUBIECTUL I (30 de puncte)

1. Grafuri Hamiltoniene

• Definiții:

- **Ciclu hamiltonian:** Un ciclu elementar care conține toate nodurile grafului exact o singură dată.
- **Graf hamiltonian:** Un graf care conține cel puțin un ciclu hamiltonian.
- **Exemple:** Ciclul simplu cu nodurile 1-2-3-4-1 este graf hamiltonian, deoarece ciclul trece o singură dată prin toate cele patru noduri. Un arbore cu cel puțin trei noduri nu este hamiltonian, deoarece nu conține niciun ciclu.
- **Teoremă de caracterizare:** Teorema lui Dirac: Dacă G este un graf neorientat cu $n \geq 3$ noduri, în care gradul fiecărui nod v satisface condiția $d(v) \geq \frac{n}{2}$, atunci G este hamiltonian.
- **Problemă (Determinare ciclu hamiltonian):** Se dă un graf neorientat cu n noduri, prin matricea de adiacență. Să se afișeze toate ciclurile hamiltoniene care pornesc din nodul 1.

Descrierea soluției: Folosim backtracking. Vectorul x reține nodurile ciclului parțial, iar viz marchează nodurile deja folosite. La pasul k , alegem un nod nevizitat adiacent cu nodul pus la pasul anterior. Când am plasat toate cele n noduri, verificăm dacă ultimul nod este adiacent cu primul; numai atunci avem ciclu hamiltonian.

• C++ (Backtracking):

```
#include <iostream>
#include <vector>
using namespace std;
int n, m, x[20];
bool viz[20], ad[20][20];
void afisare() {
    for (int i = 1; i <= n; ++i) cout << x[i] << " ";
    cout << x[1] << "\n";
}
void backtrack(int k) {
    for (int val = 1; val <= n; ++val) {
        if (!viz[val] && ad[x[k-1]][val]) {
            x[k] = val;
            viz[val] = true;
            if (k == n) {
                if (ad[val][x[1]]) afisare();
            } else backtrack(k + 1);
            viz[val] = false;
        }
    }
}
```

2. Structura sistemelor de calcul: Sistemul de operare

- **Definiție:** Un ansamblu de programe care controlează resursele hardware ale sistemului de calcul și oferă o interfață între utilizator și aplicații.
- **Funcții:**

1. Gestiunea proceselor (alocarea procesorului).
2. Gestiunea memoriei interne (RAM).
3. Gestiunea sistemului de fișiere (stocarea pe medii externe).

—

3. Programare: Secvență de elemente cu cifre egale (distinct = 1)

a) Subprogramul distinct

Subprogramul marchează cifrele care apar în scrierea lui n și returnează numărul de cifre distincte marcate. Pentru $n=0$, singura cifră prezentă este 0, deci rezultatul este 1.

Soluție C++:

```
#include <iostream>
#include <fstream>
#include <algorithm>
using namespace std;

int distinct(long long n) {
    bool ap[10] = {false};
    int count = 0;
    long long temp = n;
    if (temp == 0) return 1;
    while (temp > 0) {
        int cif = temp % 10;
        if (!ap[cif]) {
            ap[cif] = true;
            count++;
        }
        temp /= 10;
    }
    return count;
}

int main() {
    ifstream fin("DATE.TXT");
    long long x;
    int max_len = 0;
    int current_len = 0;
    while (fin >> x) {
        if (distinct(x) == 1) {
            current_len++;
        } else {
            if (current_len >= 2) {
                max_len = max(max_len, current_len);
            }
            current_len = 0;
        }
    }
    if (current_len >= 2) {
        max_len = max(max_len, current_len);
    }
    fin.close();

    if (max_len < 2) cout << "Nu exista\n";
    else cout << max_len << "\n";
}
```

```
    return 0;
}
```

Soluție Pascal:

```
program SecventeEgale;
var
    fin: text;
    x: int64;
    max_len, curr_len: integer;

function distinct(n: int64): integer;
var
    ap: array[0..9] of boolean;
    count, i, cif: integer;
    temp: int64;
begin
    for i := 0 to 9 do ap[i] := false;
    count := 0;
    temp := n;
    if temp = 0 then
        begin
            distinct := 1;
            exit;
        end;
    while temp > 0 do
        begin
            cif := temp mod 10;
            if not ap[cif] then
                begin
                    ap[cif] := true;
                    count := count + 1;
                end;
            temp := temp div 10;
        end;
    distinct := count;
end;

begin
    assign(fin, 'DATE.TXT');
    reset(fin);
    max_len := 0;
    curr_len := 0;
    while not eof(fin) do
        begin
            read(fin, x);
            if distinct(x) = 1 then
                curr_len := curr_len + 1
            else
                begin
                    if curr_len >= 2 then
                        begin
                            if curr_len > max_len then max_len := curr_len;
                        end;
                    curr_len := 0;
                end;
            end;
        end;
```

```

end;
if curr_len >= 2 then
begin
    if curr_len > max_len then max_len := curr_len;
end;
close(fin);

if max_len < 2 then writeln('Nu exista')
else writeln(max_len);
end.

```

b) Programul pentru secvența maximă

Descrierea algoritmului: Citim valorile din fișier în ordinea apariției. Pentru fiecare număr x , calculăm $\text{distinct}(x)$. Dacă rezultatul este 1, numărul are toate cifrele egale și extinde secvența curentă (curr_len). Dacă nu, secvența curentă se încheie; actualizăm maximul doar dacă lungimea ei este cel puțin 2. La final repetăm actualizarea pentru cazul în care cea mai lungă secvență se termină chiar la sfârșitul fișierului.

Eficiență: Fiecare număr este citit o singură dată, iar testarea cifrelor are cel mult 10 pași pentru valori până la 10^9 . Algoritmul este liniar în numărul termenilor din fișier și folosește memorie constantă.

SUBIECTUL al II-lea (30 de puncte)

1. Aspecte ale proiectării didactice pentru conținutul A: Structuri repetitive

a) Tipuri de lecție și momente ale lecției

Tipuri de lecție:

1. Lecție de comunicare/însușire de noi cunoștințe.
2. Lecție de formare de priceperi și deprinderi.
3. Lecție de recapitulare și sistematizare.

Momente esențiale ale lecției:

1. Organizarea clasei și captarea atenției.
2. Anunțarea temei și a obiectivelor.
3. Dirijarea învățării prin explicații, exemple și exerciții.
4. Fixarea, evaluarea și formularea temei pentru acasă.

b) Activități de învățare

Pentru capitolul **Structuri repetitive**, pot fi folosite următoarele activități:

1. Identificarea situațiilor în care este necesară repetarea unui bloc de instrucțiuni.
2. Compararea structurilor `while`, `do...while` și `for`.
3. Implementarea unui algoritm de calcul al sumei cifrelor unui număr.
4. Testarea programului pentru cazuri limită: număr cu o singură cifră, număr terminat în zero, număr mare.

Scenariu pentru activitatea 3:

- Profesorul propune problema: „Se citește un număr natural. Să se determine suma cifrelor sale.”

- Profesorul întreabă ce operații permit extragerea ultimei cifre și eliminarea acesteia. Elevii identifică $n \bmod 10$ și $n \div 10$.
- Profesorul construiește împreună cu elevii structura repetitivă: cât timp $n > 0$, se adaugă ultima cifră la sumă și se elimină cifra.
- Elevii implementează algoritmul, rulează exemple și verifică manual rezultatele.

Soluție C++:

```
#include <iostream>
using namespace std;

int main() {
    int n, s = 0;
    cin >> n;
    while (n > 0) {
        s += n % 10;
        n /= 10;
    }
    cout << s;
    return 0;
}
```

2. Test oral pentru conținutul A: Structuri repetitive

Item	Enunț	Răspuns așteptat
1	Ce este o structură repetitivă?	O instrucțiune care execută repetat un bloc cât timp o condiție este îndeplinită sau pentru un număr determinat de pași.
2	Când este recomandată folosirea instrucțiunii for?	Când numărul de repetări este cunoscut sau determinabil înaintea buclei.
3	Ce diferență există între while și do...while?	while testează condiția înainte de execuție, do...while după execuție, deci execută blocul cel puțin o dată.
4	Ce se întâmplă dacă într-o buclă while condiția nu devine niciodată falsă?	Apare o buclă infinită.
5	Dați un exemplu de problemă rezolvată cu repetare.	Suma cifrelor unui număr, numărarea divizorilor, parcurgerea unui vector etc.

SUBIECTUL al III-lea (30 de puncte)

Învățarea: concept, condiții interne și condiții externe

Concept: Învățarea este procesul psihopedagogic prin care individul dobândește cunoștințe, deprinderi, capacități, atitudini și comportamente noi, pe baza experienței și exercițiului.

În școală, învățarea este organizată, dirijată și evaluată, având ca scop dezvoltarea competențelor elevilor.

Condiții interne ale învățării:

- motivația elevului și interesul pentru sarcină;
- nivelul de dezvoltare intelectuală și cunoștințele anterioare;
- atenția, memoria, capacitatea de concentrare;
- stilul de învățare și încrederea în propriile posibilități;
- starea afectivă și disponibilitatea pentru efort.

La informatică, un elev care stăpânește operațiile aritmetice de bază și are motivație pentru rezolvarea problemelor va înțelege mai ușor structurile repetitive.

Condiții externe ale învățării:

- calitatea explicației profesorului;
- metodele didactice folosite;
- organizarea clasei și climatul educațional;
- mijloacele de învățământ: calculator, proiector, fișe de lucru, platforme online;
- timpul alocat exersării și feedbackul primit.

Învățarea eficientă apare prin corelarea condițiilor interne cu cele externe. Profesorul adaptează sarcinile la nivelul elevilor, oferă exemple gradate, creează situații de exersare și verifică permanent înțelegerea.

II. Rezolvare Definitivat Informatică de gestiune Varianta 1 (14 iulie 2014)

[Subiect PDF](file:

SUBIECTUL I (30 de puncte)

1. Imprimanta în arhitectura sistemului de calcul

Imprimanta este un dispozitiv periferic de ieșire. Ea primește date de la sistemul de calcul și le transpune pe suport fizic, de regulă hârtie. Comunicarea cu sistemul se face prin interfață USB, rețea sau conexiune wireless, iar sistemul de operare o controlează prin driver.

Tipuri de imprimante:

- **matricială**: tipărește prin lovirea unei benzi tușate cu ace; este robustă, dar zgomotoasă și cu rezoluție redusă;
- **inkjet**: pulverizează picături fine de cerneală; este potrivită pentru imagini color și documente uzuale;
- **laser**: folosește toner și tambur fotosensibil; are viteză mare și cost bun pentru volum ridicat.

Parametri de performanță:

- **rezoluția**, exprimată în dpi, influențează claritatea textului și a imaginilor;
- **viteza de tipărire**, exprimată în pagini pe minut, determină eficiența la volume mari.

2. Sistemul de operare

Sistemul de operare este ansamblul de programe care administrează resursele hardware și software ale calculatorului și oferă interfață între utilizator, aplicații și echipamente.

Funcții:

- gestiunea proceselor și alocarea procesorului;
- gestiunea memoriei interne;
- administrarea fișierelor și directoarelor;
- controlul dispozitivelor periferice și al driverelor;
- asigurarea interfeței cu utilizatorul.

3. Subprogramul distinct și programul cerut

Antet C++:

```
int distinct(long long n);
```

Program C++:

```
#include <iostream>
#include <fstream>
using namespace std;

int distinct(long long n) {
    bool ap[10] = {false};
    int nr = 0;
    do {
        int c = n % 10;
        if (!ap[c]) {
            ap[c] = true;
            nr++;
        }
    }
```



```

        n /= 10;
    } while (n > 0);
    return nr;
}

int main() {
    ifstream fin("date.txt");
    long long x;
    for (int i = 1; i <= 7; ++i) {
        fin >> x;
        if (distinct(x) == 1)
            cout << x << " ";
    }
    fin.close();
    return 0;
}

```

Explicație: un număr are toate cifrele egale exact atunci când numărul cifrelor distincte din scrierea lui este 1. Se citesc cele 7 valori și se afișează numai cele care îndeplinesc această condiție.

4. Bază de date pentru companie de telefonie

Tabele:

- CLIENT(id_client, nume, prenume, adresa);
- TIP_ABONAMENT(id_tip, denumire, pret, minute_nationale);
- NUMAR_TELEFON(id_numar, numar, id_client, id_tip, data_activare, data_inchidere).

Relații:

- CLIENT 1:M NUMAR_TELEFON;
- TIP_ABONAMENT 1:M NUMAR_TELEFON.

Restricții:

- numar unic;
- pret >= 0, minute_nationale >= 0;
- data_inchidere nulă pentru numere active sau mai mare decât data_activare;
- cheile străine id_client și id_tip trebuie să existe.

Interogări:

```

-- clienții care dețin un anumit tip de abonament
SELECT c.nume, c.prenume, c.adresa, n.numar
FROM CLIENT c
JOIN NUMAR_TELEFON n ON n.id_client = c.id_client
WHERE n.id_tip = :id_tip
      AND n.data_inchidere IS NULL;

-- clienții care dețin în prezent cel puțin două numere
SELECT COUNT(*) AS nr_clienti
FROM (
    SELECT id_client
    FROM NUMAR_TELEFON
    WHERE data_inchidere IS NULL
    GROUP BY id_client
    HAVING COUNT(*) >= 2
) q;

```

```
-- abonamente nesolicitate în ultimul an
SELECT t.*
FROM TIP_ABONAMENT t
LEFT JOIN NUMAR_TELEFON n
  ON n.id_tip = t.id_tip
  AND n.data_activare >= CURRENT_DATE - INTERVAL '1 year'
WHERE n.id_numar IS NULL;
```

SUBIECTUL al II-lea (30 de puncte)

1. Proiectare didactică pentru structuri repetitive

Tipuri de lecție: lecție de comunicare de cunoștințe, lecție de formare de priceperi și deprinderi, lecție de recapitulare/sistematizare.

Momente esențiale: organizarea clasei, captarea atenției, anunțarea temei și obiectivelor, dirijarea învățării, fixarea și evaluarea.

Activități de învățare pentru capitolul **Structuri repetitive**:

1. identificarea situațiilor în care este necesară repetarea;
2. compararea instrucțiunilor while, do...while, for;
3. implementarea sumei cifrelor unui număr;
4. testarea programului pentru valori limită.

Scenariu pentru activitatea 3: profesorul propune problema sumei cifrelor, discută operațiile $n \% 10$ și $n / 10$, construiește cu elevii bucla while ($n > 0$), apoi elevii implementează și rulează programul. Profesorul verifică rezultatele și discută erorile: inițializare greșită a sumei, condiție de oprire incorectă, uitarea eliminării ultimei cifre.

2. Test oral pentru structuri repetitive

Item	Enunț	Răspuns așteptat
1	Definiți structura repetitivă.	Execută repetat un bloc de instrucțiuni în funcție de o condiție sau de un contor.
2	Când folosim for?	Când numărul de pași este cunoscut sau determinabil.
3	Ce diferență există între while și do...while?	while testează înainte, do...while după execuția blocului.
4	Ce produce o condiție mereu adevărată?	O buclă infinită.
5	Dați un exemplu de algoritm cu repetare.	Suma cifrelor, numărarea divizorilor, parcurgerea unui vector.

SUBIECTUL al III-lea (30 de puncte)

Învățarea este procesul prin care elevul dobândește cunoștințe, deprinderi, capacități, atitudini și comportamente, pe baza exercițiului și experienței.

Condiții interne: motivația, atenția, memoria, nivelul cunoștințelor anterioare, stilul de învățare, starea afectivă și capacitatea de efort.

Condiții externe: calitatea explicației profesorului, metodele didactice, climatul clasei, mijloacele de învățământ, timpul de lucru, feedbackul și organizarea sarcinilor.

În informatică, învățarea eficientă apare când elevul are baza logică necesară și motivație, iar profesorul oferă exemple gradate, activitate practică la calculator, feedback imediat și situații variate de exersare.